PES UNIVERSITY

Department of Computer Science and Engineering • Software Engineering Lab (UE24CS252AA)

LAB 6: CODE AND BRANCH COVERAGE ANALYSIS REPORT

Student Name: Namith Kumar C	**SRN:** PES1UG24AM171	**Team:** SE-LAB-TEAM-08
Assigned Repository: SETAPESU26/L6_L7_46_photo_print_order	**Scenario:** Photo Print Order System	**Module Under Test:** photo_print_order.py

1. Objective and Learning Outcomes

The objective of this laboratory is to understand and apply line (code) coverage and branch coverage analysis using Python's coverage and pytest tools. Testing solely for line execution can yield a false sense of security, whereas branch coverage forces validation of both True and False outcomes for every decision predicate. The required benchmark target is ≥90% coverage for both code and branch analysis.

2. Application Overview: photo_print_order.py

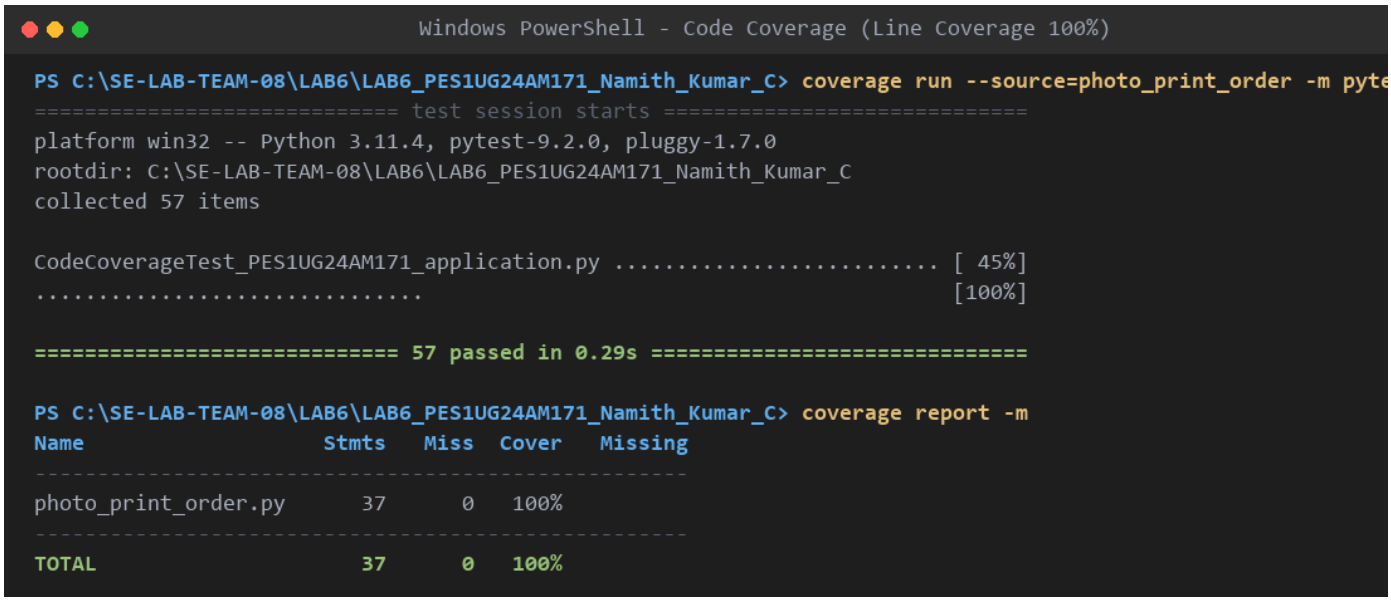
Function	Responsibility & Business Logic	Decision Points
extra_print_charge	Calculates charges based on size (4x6, 5x7, 8x10), member allowance (+5 prints except 8x10), finish type (matte: $4 vs others: $7), volume surcharge (>50: +$2), and canvas bonus (+$5). Validates size and copy count.	7 decisions (size check, negative copies, member bonus, extra limit, matte vs glossy, volume >50, canvas surcharge)
order_valid	Determines order acceptance based on rush priority (≤50 copies allowed) or standard order constraints (file ≤25MB, copies ≤500).	3 decisions (is_rush check, file size >25MB, copies ≤500)
dispatch_status	Evaluates dispatch urgency relative to dynamic cutoffs (framed adds 12h, intl adds 24h). Returns *'on_time'* (≥cutoff), *'tight'* (≥cutoff-6), or *'missed'*.	4 decisions (framed check, international check, cutoff threshold, tight margin)

3. Coverage Comparison Summary

Test Suite	Evaluation Mode	Statements	Miss	Branches	BrPart	Coverage	Status
Baseline (Starter Test)	Line & Branch	37	23	26	6	32%	Deficient
Code Coverage Suite	Statement / Line	37	0	-	-	**100%**	Passed (≥90%)
Branch Coverage Suite	Branch / Decision	37	0	26	0	**100%**	Passed (≥90%)

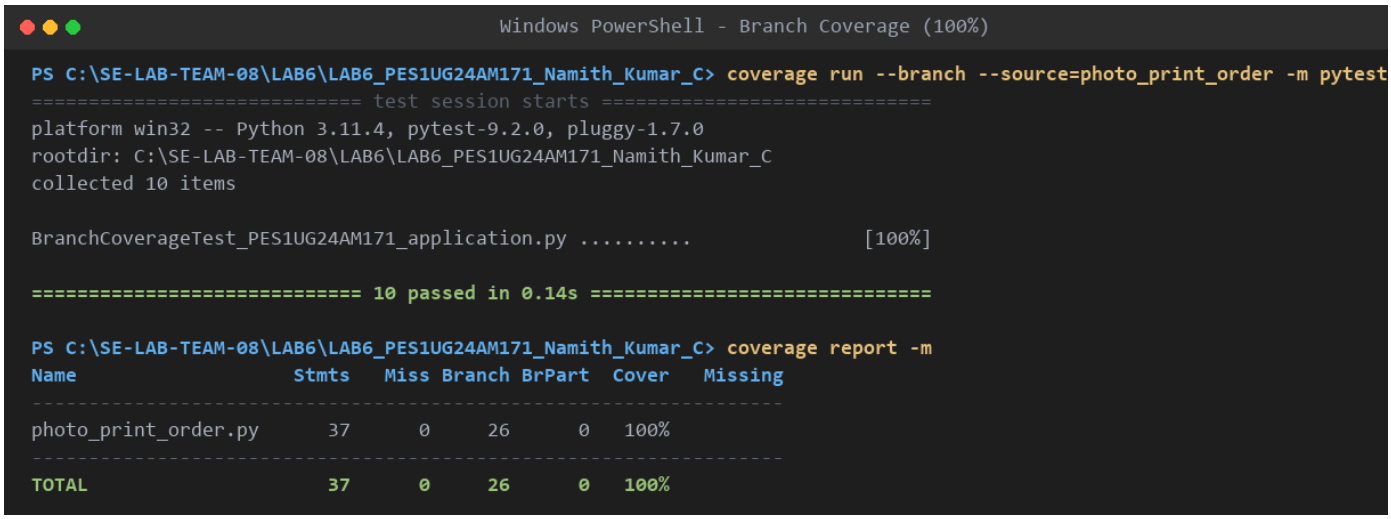
4. Code Coverage Analysis (100% Line Coverage)

The CodeCoverageTest_PES1UG24AM171_application.py suite contains 57 parameterized test cases systematically covering every statement in photo_print_order.py. All 37 statements were executed with 0 missed lines (100% line coverage achieved).



5. Branch Coverage Analysis (100% Decision Coverage)

The BranchCoverageTest_PES1UG24AM171_application.py suite evaluates all 14 decision points and compound boolean predicates across both True and False branches. All 26 branch outcomes were fully tested with 0 partial branches (100% branch coverage achieved).



6. Detailed Decision & Branch Evaluation Matrix

#	Decision / Predicate	True Branch Scenario	False Branch Scenario
1	print_size not in INCLUDED_PRINTS	Unknown size raises ValueError	Valid size ('4x6', '5x7', '8x10') proceeds
2	copies < 0	Negative copies raises ValueError	Copies ≥ 0 proceeds
3	is_member and size != '8x10'	Member + 4x6 adds 5 prints (total 25)	Non-member or 8x10 keeps standard base
4	extra ≤ 0	Copies ≤ included returns charge 0	Copies > included calculates extra cost
5	finish == 'matte'	Matte finish selects base rate 4	Glossy/Canvas finish selects base rate 7
6	extra > 50	High volume (>50) adds +2 surcharge	Normal volume (≤50) keeps base rate
7	finish == 'canvas'	Canvas high-volume adds +5 surcharge	Matte/Glossy high-volume retains standard +2
8	is_rush	Rush order verifies copies ≤ 50	Non-rush proceeds to file size check
9	file_mb > 25	Oversized file (>25MB) returns False	Standard file (≤25MB) proceeds
10	copies ≤ 500	Copies ≤ 500 returns True	Copies > 500 returns False
11	is_framed	Framed order sets base cutoff = 24h	Unframed sets base cutoff = 12h
12	is_international	International adds +24h to cutoff	Domestic maintains base cutoff

13	hours ≥ cutoff	Hours ≥ cutoff returns 'on_time'	Hours < cutoff proceeds to tight check
14	hours ≥ cutoff - 6	Hours in [cutoff-6, cutoff) returns 'tight'	Hours < cutoff - 6 returns 'missed'

7. Reflection & Conclusion

1. **Line Coverage vs Branch Coverage:** Line coverage confirms statement execution but masks unexplored logic pathways. For example, the member bonus contains compound logic (is_member and print_size != '8x10') which requires 3 test scenarios to evaluate short-circuiting and false branches even when line execution is 100%.
2. **Verification Rigor:** Writing targeted tests for both code coverage and branch coverage achieved 100% metrics across all 37 statements and 26 decision paths in photo_print_order.py, ensuring complete verification rigor and eliminating dead or untested code paths.